◆ DRAGONAI · 烛龙智元

AI FDE 工程师就业实训营

学生讲义

Day 2

前端与 Skill 机制

学会调动 AI 完成真实的前端任务

杭州烛龙智元科技有限公司

目录

学习目标 1

01 开场：为什么选择 Vibe Coding 1

1.1 课程节奏与路线 1

1.2 指挥 AI 的能力 2

1.3 新的学习入口 2

1.4 开发搭档与实习生 3

1.5 好问题与好工具 3

02 前端最小地图 4

2.1 前端与后端 4

2.2 前后端的配合 4

2.3 前端的多种形态 5

2.4 WebApp 的用途 5

2.5 网页的三层结构 6

2.6 阅读 AI 写的代码 6

2.7 浏览器开发者工具 7

03 Vibe Coding 工具链 8

3.1 IDE 与 CLI 8

3.2 开发工具组合 8

3.3 Node 的作用 9

3.4 安装与检查 Node 10

3.5 终端与常用命令 11

3.6 Claude Code 安装检查 11

3.7 cc switch 与模型配置 12

3.8 第一次 AI 对话 12

04 和 AI 对话的标准姿势 13

4.1 坏 Prompt 的问题 13

4.2 好 Prompt 的结构 13

4.3 概念与报错的提问 14

4.4 先规划后执行 15

4.5 提示词工作流 15

05 第一次前端实操：个人主页 15

5.1 实操目标与流程 15

5.2 描述页面与生成代码 16

5.3 预览与检查 16

5.4 继续对话修改 16

5.5 页面验收清单 16

5.6 理解代码结构 17

06 前端框架入门 17

6.1 组件与状态 18

6.2 框架的选择 18

6.3 Vue 与项目初始化 18

6.4 React 与入口文件 19

6.5 Next.js 与框架对照 19

6.6 读懂项目目录 20

07 界面生成与 Todo 项目 20

7.1 界面初稿与快速原型 20

7.2 好页面 Prompt 的要素 21

7.3 Todo 网页的迭代 22

7.4 版本回滚 22

08 Claude Code 进阶工作流 22

8.1 Claude Code 的工作模式 23

8.2 终端命令与后台任务 23

8.3 上下文与项目记忆 23

8.4 Hook 与 Sub Agent 24

09 Skill 机制 24

9.1 Skill 的含义 24

9.2 Skill 的作用与本质 24

9.3 Skill 的目录与内容 25

9.4 Skill 与相近概念 26

9.5 前端 Skill 的拆解与应用 26

10 总结与课后练习 27

10.1 验收与常见错误 27

10.2 完整的工作流 28

练习与作业 **29**

复习题 29

课后作业 29

学习目标

第二天学习用 Vibe Coding 的方式调动 AI 完成前端任务，在 AI 的协助下做出个人主页与 Todo 网页，并把做对的方法写成可复用的 Skill。

1. 说明 AI 在 Vibe Coding 中作为开发搭档的角色，并按提示词工作流的五个步骤写出一段完整的提示词。
2. 区分前端与后端，说明 HTML，CSS 与 JavaScript 各自负责的部分，并用浏览器开发者工具查看页面结构，报错与请求。
3. 完成 Node 与 Claude Code 的安装检查和 cc switch 的模型配置，在项目目录里用 npm 命令运行前端项目。
4. 完成一个个人主页和一个能新增，完成和删除任务的 Todo 网页，并用验收清单判断页面能否交付。
5. 区分 Vue3，React 与 Next.js 的定位，说明 Claude Code 对话，规划与执行三种模式各自适用的场合。
6. 说明 Skill 的作用与目录结构，区分 Skill 与 Prompt，完成一个可复用的 Skill 雏形。

01
开场：为什么选择 Vibe Coding

1.1 课程节奏与路线

第二天的学习节奏是先会用，再会改，最后把经验固定下来。拆开来是五个动作：

动作	含义
“会问”	能把问题向 AI 说清楚
“会读”	能读懂 AI 写的代码
“会跑”	能把项目在本机运行起来
“会改”	能通过继续对话逐步修改
“会沉淀”	能把做对的方法整理成可复用的 Skill

五个动作服务于同一个结果：学员能够独立启动并推进一个前端任务。当天的内容按五段推进：

1. 对话方法：怎样向 AI 提出清楚的要求。
2. 前端地图：前端由哪些部分组成。

3. 工具环境：开发工具的安装与配置。
4. 页面实操：做出个人主页与 Todo 网页。
5. Skill 机制：把做对的方法写成 Skill。

学完之后，学员能掌握 Claude Code 的使用方法，并做出三样东西：

- 一个能交互的 Todo 网页。
- 一个用 Skill 美化过的 Todo 网页。
- 一个可复用的 Skill 雏形。

贯穿全程的原则是：每一步都可以让 AI 参与，但每一步都要由人来验收。

1.2 指挥 AI 的能力

第二天的重点不是记语法，而是学会调动 AI 完成真实的前端任务。一个人能做成多少事，越来越取决于他能调动多少 AI 协同工作。

AI 可以写很多代码，但决定方向，判断对错和验收结果仍然要靠人。因此 Vibe Coding 真正要学的是指挥 AI。

工具会变，工作流会留下。需要记住的是描述问题与验证结果的方法，以及把经验复用的习惯。学习时不背语法，不堆概念，以实操为主。掌握了工作流，换用其他工具时这套方法同样适用。

1.3 新的学习入口

不会就问 AI，是一种新的学习入口。不懂的概念与命令，看不懂的报错与代码结构，都可以先问 AI。只问一次往往不够，要追问到自己能复述为止。

以终端里的几条基础命令为例：

```
pwd
ls
cd
mkdir
...
```

不认识这些命令时，先问 AI 每条命令的作用。四条命令的作用分别是：

命令	作用
`pwd`	显示当前所在的目录
`ls`	列出当前目录里的文件
`cd`	切换到另一个目录
`mkdir`	新建一个目录

Windows 用户在 PowerShell 里同样可以使用这四条命令。能用自己的话说出每条命令的作用，才算真正学会了这些命令。

1.4 开发搭档与实习生

在 Vibe Coding 中，AI 不是老师，也不是搜索框，而是**开发搭档**。方向与验收由人掌握，具体的开发工作交给 AI 完成，双方一起推进任务。

搜索框和开发搭档的用法不同。用搜索框时，学员输入关键词，搜索框返回链接，再由学员自己筛选。和开发搭档合作时，学员给出目标并补充上下文，再和 AI 一起迭代。

和开发搭档合作，可以借用一个比喻：把 AI 当实习生。实习生很勤快，但需要交代清楚做什么与为什么做，不能碰哪里与怎么验收。落到提问里，就是三条做法：

做法	具体要求
说清结果	先描述要交付的页面或功能，说明给谁用，做到什么程度，不要只说“帮我做”
交代现场	告诉 AI 项目现在的状态与已有文件，附上报错信息，说明约束条件
给出边界	说明哪里不要改，怎样检查算完成

三条做法合在一起，构成一次完整的任务交代。

1.5 好问题与好工具

好问题比好工具更重要。以个人主页为例，一个好问题可以这样写：

> 我想做一个个人主页。目标用户是招聘面试官。请先给我页面结构，不要直接写代码。

好问题包含三部分：先说明要交付什么，再明确不做大范围修改，不随意添加依赖，最后要求 AI 给出检查方式。例子交代了交付物与目标用户，并要求 AI 先给页面结构，不直接写代码。实际提问时再补上后两部分，问题才完整。

先要结构，再要代码，返工会少很多。

小结

Vibe Coding 的核心能力是指挥 AI，人负责方向与验收，AI 负责具体的开发工作。工具会更新换代，描述问题，验证结果和复用经验的方法可以一直沿用。遇到不会的内容先向 AI 提问，直到能用自己的话讲出来。与 AI 协作时按交代实习生的方式提要求，交付物，现场情况和边界都要讲到。提问时先要页面结构，确认之后再要代码。

02
前端最小地图

2.1 前端与后端

动手修改页面之前，先分清改动落在界面，逻辑，还是数据上。为此先看前端与后端各自负责什么。

前端负责用户能看见，能点击，能感受到的部分，把数据和功能变成可以使用的界面。**后端**负责数据与权限，也负责业务逻辑和服务的稳定性，让前端背后的系统真正运转起来。两者的分工可以从五个方面看：

部分	方面	内容
前端	看得见	页面内容与布局，以及按钮和表单，都属于前端的表达范围
前端	能操作	用户通过点击与输入，切换与提交完成动作
前端	会反馈	加载中，成功与失败，以及没有数据时的空状态，都要给出提示，这决定体验是否可信
后端	处理规则	登录权限，业务判断与数据校验都在后端完成
后端	管理数据	数据库与文件，日志与缓存，让系统能长期运行

前端面向用户的操作与感受，后端面向数据，接口和业务规则。

2.2 前后端的配合

前端和后端通过 **API** 配合。API 是请求与响应的约定，前端按约定发出请求，后端按约定返回响应。一次完整的交互分六步：

1. 用户点击按钮或输入内容。
2. 前端收集输入，向后端发起请求，常见的请求有登录与查询，提交与上传。
3. 后端的接口层接收请求，再完成业务逻辑与权限校验，并处理数据。
4. 后端读取或写入数据库，数据库保存用户和业务数据。
5. 后端把处理结果返回给前端。
6. 前端根据返回的数据更新页面，用户看到新的内容。

后端还会连接支付，短信与地图这类第三方服务。前端开发者不能只关心“美化页面”，还要理解数据从哪里来，以及用户的每个动作会触发什么请求。

2.3 前端的多种形态

前端不只是网页。客户端和 WebApp，移动端和小程序，都是把功能包装成用户能操作的界面，所以前端是一组形态，不是一种单一页面。图中左边是电脑上的一个桌面客户端，右边是手机上的应用界面，二者和浏览器里打开的网页一样，都属于前端。

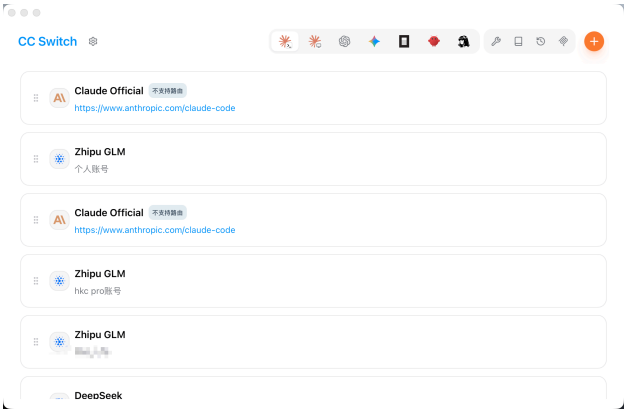


图 1　桌面客户端，图中是 cc switch 的窗口。



图 2　手机上的应用界面。

形态虽然不同，要解决的问题相同：用户能看见什么，能操作什么，能得到什么反馈。

2.4 WebApp 的用途

WebApp 指在浏览器里打开就能使用的应用，是第二天练习的主要对象。WebApp 门槛低，反馈快，作品便于展示，适合初学者练习 Vibe Coding。

前端开发是把想法变成作品的捷径，做出的页面可以展示，试用和传播。图中是两个用前端做出的作品：左边是一个任务平台的首页，用户可以在页面上发布任务与接单，也可以接入自己的智能体。右边是一个由粒子组成的动画页面，左侧面板可以切换图案与粒子颜色。



图 3　任务平台的首页。



图 4　粒子组成的动画页面。

WebApp 常见的用途有四类：

- 官网：把信息讲清楚。
- 个人主页：展示自己是谁。

- 产品原型：验证一个想法。
- 个人助手：把流程自动化。

页面能打开，能试用，想法才能被别人看见。

2.5 网页的三层结构

网页由三层组成：HTML 是页面的骨架，CSS 决定页面的样子，JavaScript 负责页面的行为。用房子打比方，三层的分工是：

层	比喻	负责	例子
HTML	房子的结构	定义页面上有哪些内容	页面标题和登录表单
CSS	房子的装修	控制内容的颜色，字体与布局	按钮变蓝和卡片阴影
JavaScript	房子的电器和开关	处理点击与输入，请求数据并更新页面	点击按钮弹出提示和表单校验

图中上半部分用房子比喻三层的分工，下半部分是一个示意页面：HTML 提供标题与图片，按钮与输入框，CSS 美化样式，JavaScript 增加交互，三者合起来组成一个完整的网页。



图 5　HTML，CSS 与 JavaScript 的分工。

记忆口诀是“HTML 搭骨架，CSS 做美化，JavaScript 管交互”，学习顺序也是先结构，再样式，最后交互。

2.6 阅读 AI 写的代码

阅读 AI 写的代码，不需要逐字逐句地读。先看文件名，再看整体结构，最后看关键函数，不要一开始就陷进每一行语法。阅读时抓住四件事：

1. 页面入口在哪里。

2. 组件怎么拆。

3. 样式放在哪里。

4. 用户操作触发了什么逻辑。

其中的组件，指页面里可以拆开单独维护的零件，例如按钮与卡片，列表与表单。动手处理 AI 写的项目时，按三步走：

1. 先运行起来：项目能启动，能预览，再进入下一轮优化。

2. 再读结构：从 `package.json`，入口文件和组件目录开始看。`package.json` 记录项目的名称，依赖和常用命令。

3. 小步修改：一次只改一个方向，避免大面积返工。

先理解结构，再交给 AI 执行。

2.7 浏览器开发者工具

在浏览器里按 F12 打开的开发者工具，相当于前端开发者的显微镜。按 F12 没有反应时，也可以在页面上点右键，选择“检查”。图中红框圈出的是面板标签，Elements，Console 与 Network 都在这一排，当前选中的是 Network 面板。

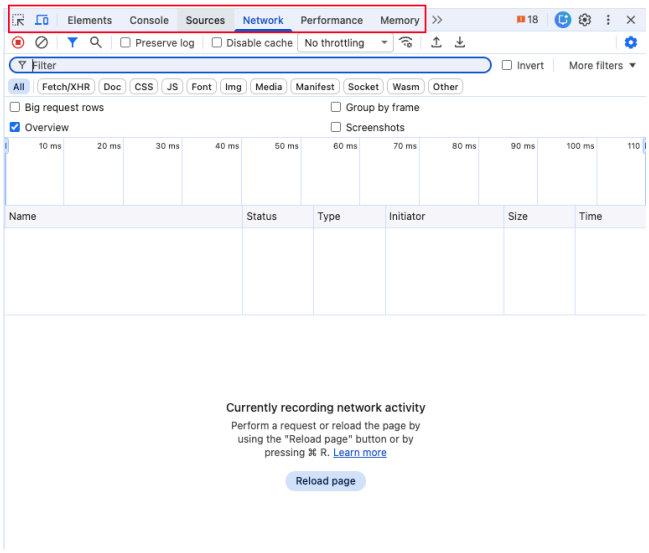


图 6　浏览器开发者工具，红框里是各个面板的标签。

开发者工具里重点看三个面板：

面板	查看内容
Elements	页面的 HTML 结构和样式
Console	JavaScript 的报错和输出
Network	页面发出的每一个请求，包括前端与后端之间的请求和响应

页面出了问题，先在这三个面板里分别查看结构，报错和请求。

小结

修改页面之前，先判断改动属于界面，逻辑还是数据。前端面向用户的观看，操作与反馈，后端处理规则和数据，两者通过 API 交换请求与响应。前端形态多样，WebApp 因为门槛低，反馈快而成为初学者的练习对象。网页由 HTML，CSS 与 JavaScript 三层构成，阅读 AI 写的代码时抓住入口与组件，样式与交互逻辑这四件事。排查页面问题时，开发者工具的三个面板分别对应结构，报错与请求。

03
Vibe Coding 工具链

3.1 IDE 与 CLI

工具只是手段，目的是让整条工作流顺利运行起来。AI 编程工具有两种使用方式：**IDE** 和 **CLI**。IDE 是带图形界面的集成开发环境，CLI 是在终端里输入命令的命令行界面。

方式	特点
IDE	图形界面友好，适合查看项目，适合初学者
CLI	命令更直接，适合自动化和深度开发

两种方式各有适用的场合，选择时要看任务与成本，也要看上下文和验收方式。第二天的练习把两种方式搭配起来使用。

3.2 开发工具组合

第二天的工具组合是 Claude Code 加上 VsCode 或 Cursor。Claude Code 是主力开发助手，在终端里运行。VsCode 和 Cursor 属于 IDE，用来查看项目文件。

作为项目级的 AI Coding 工具，Claude Code 适合在项目目录里通过对话修改代码，运行命令，并不断修复问题，用于项目内的持续开发。图中 Claude Code 正在按对话要求往项目目录里写入一个网页文件，界面底部是继续输入指令的地方。

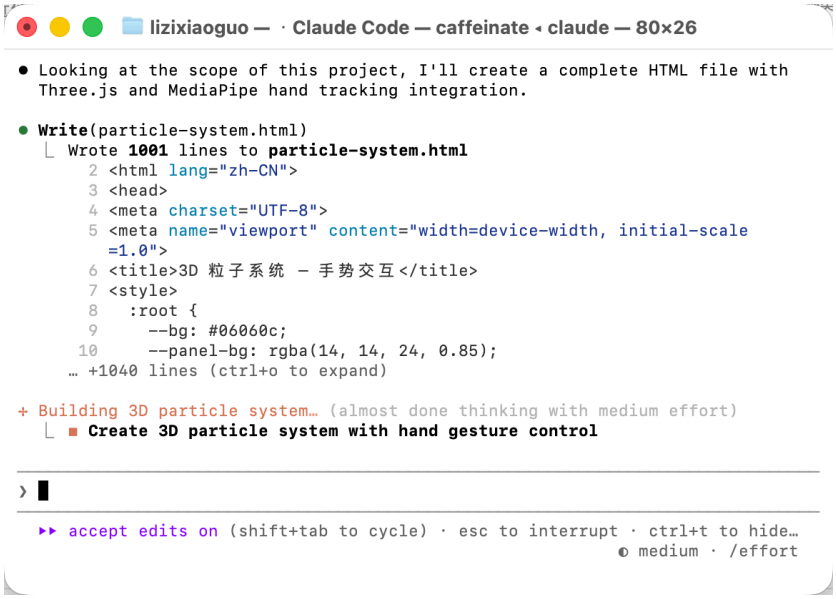


图 7　Claude Code 在项目目录里写入网页文件。

Codex 是另一款 AI Coding 工具，适合让 AI 读项目，做修改，产出可以验证的结果。这种工作流把任务，修改和验证连成一个完整的循环。

两款工具有同一条注意事项：生成的结果必须在本地运行验证。

3.3　Node 的作用

Node 是 JavaScript 的运行环境，让 JavaScript 可以脱离浏览器运行。打个比方，Node 把 JavaScript 从网页里带到了服务器上：浏览器里的 JavaScript 负责页面交互，Node 里的 JavaScript 负责服务端程序。图中用四块内容介绍 Node。

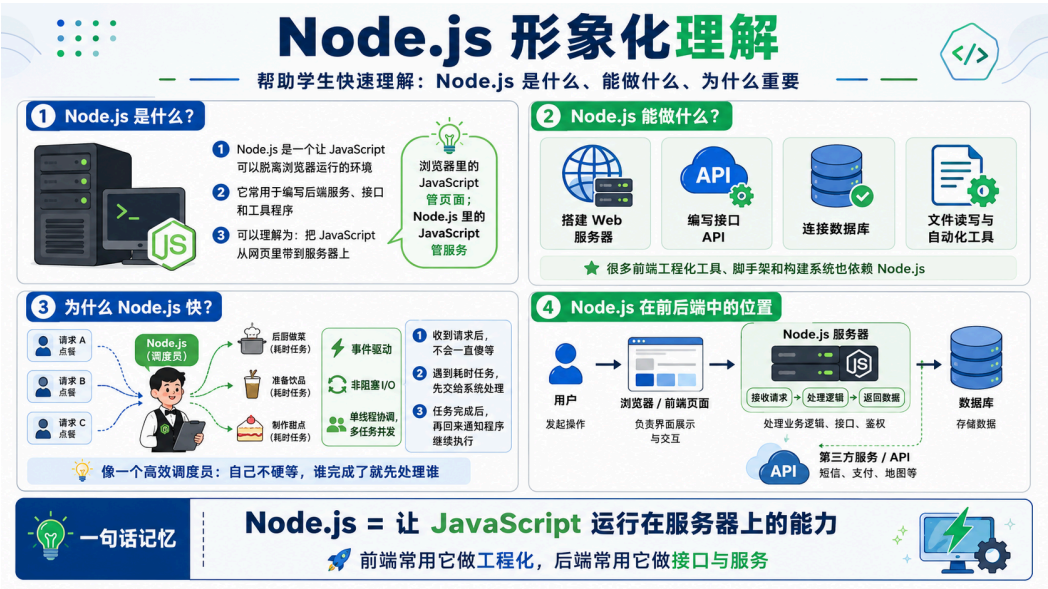


图 8　Node 的作用，处理方式与在前后端中的位置。

Node 能做的事有四类：

- 搭建 Web 服务器。
- 编写接口 API。
- 连接数据库。
- 文件读写与自动化工具。

很多前端工程化工具，脚手架和构建系统也依赖 Node。所以前端开发常用 Node 做工程化，后端开发常用 Node 写接口与服务。

Node 处理请求的速度快，原因在于事件驱动与非阻塞 I/O。图中左下用餐厅点餐作比：几位顾客同时点餐，调度员把做菜，准备饮品和制作甜点这些耗时任务分派出去，不在原地等待，哪项任务先完成就先处理哪项。对应到程序里，Node 收到请求后不会停下来等待，遇到耗时任务先交给系统处理，任务完成后再通知程序继续执行。

图中右下是 Node 在前后端中的位置：用户在浏览器页面上发起操作，Node 服务器接收请求，处理逻辑并返回数据，数据存放在数据库里。需要时，Node 服务器还会调用短信，支付与地图等第三方服务。

前端项目的依赖安装，启动与构建都要用到 Node，搭建开发环境的第一件事就是安装 Node。

3.4 安装与检查 Node

在 MacOS 上用 brew 安装 Node，装好后查看 Node 和 npm 的版本号，能显示版本号就说明安装成功：

```
$ brew install node
$ node -v
$ npm -v
```

npm 随 Node 一起安装，前端项目用 npm 安装依赖和运行命令。Windows 上可以从 Node 官网下载安装包安装，装好后在 PowerShell 里执行同样的两条检查命令。

安装之后还要检查环境：

```
$ which node
$ which npm
$ npm create vite@latest
```

`which node` 和 `which npm` 显示终端实际调用的 node 与 npm 程序所在的路径。同一台电脑上可能装过不止一份 Node，这一步能确认用的是哪一份。Windows 的 PowerShell 里用 `Get-Command node` 查看同样的信息。`npm create vite@latest` 会用 Vite 新建一个前端项目，能顺利执行，说明 Node 和 npm 都可以正常工作。

检查比安装更重要，开发者要确认当前命令实际指向的程序。

3.5 终端与常用命令

终端是用文字控制电脑的工具，不是黑魔法。学员不需要记住全部命令，遇到不会的先问 AI。在终端里执行命令遵循三条规则：

1. 先定位。用 `pwd` 确认当前目录，用 `ls` 查看目录里的文件。命令的作用范围通常由当前目录决定，所以先确认所在目录，再执行下一步。
2. 看完整输出。终端日志比最后一行更重要，报错的原因常常出现在前面的输出里。
3. 报错交给 AI。命令执行失败时，把完整报错连同执行的命令一起交给 AI。

查看位置与切换目录的常用命令：

```
$ pwd
$ ls
$ cd your-project
```

`cd your-project` 进入项目目录，`your-project` 要换成实际的项目文件夹名。

前端项目的常用命令：

```
$ npm install
$ npm run dev
$ npm run build
```

`npm install` 安装项目依赖，`npm run dev` 启动开发服务器供本地预览，`npm run build` 把项目构建成可以发布的文件。遇到不熟悉的命令，执行前可以先让 AI 解释这条命令的作用。

3.6 Claude Code 安装检查

Claude Code 安装好之后，先在终端里执行两条命令：

```
$ claude --version
$ claude
```

`claude --version` 查看版本号，`claude` 在当前目录打开 Claude Code 的对话。Windows 用户在 PowerShell 里执行这两条命令。对话打开后，在输入框里输入：

```
/help
```

`/help` 用来查看帮助。以斜杠开头的命令属于 Claude Code 对话内的命令，不在终端里执行。

能打开对话，不代表项目环境已经配置好，Node，npm 与模型配置还要分别确认。

3.7　cc switch 与模型配置

cc switch 是在不同模型与 API 配置之间切换的工具，可以减少手动修改配置带来的混乱。让 Claude Code 通过 cc switch 使用 DeepSeek，分四步：

1. 确认 API Key。
2. 写入配置。
3. 切换模型。
4. 发起一次测试对话。

第一步要用的 API Key 在 DeepSeek 的 API 开放平台获取。第二步在 cc switch 里添加新供应商，先选择一个预设的供应商，再在下方填写 API Key 等字段，图中是添加新供应商的界面。

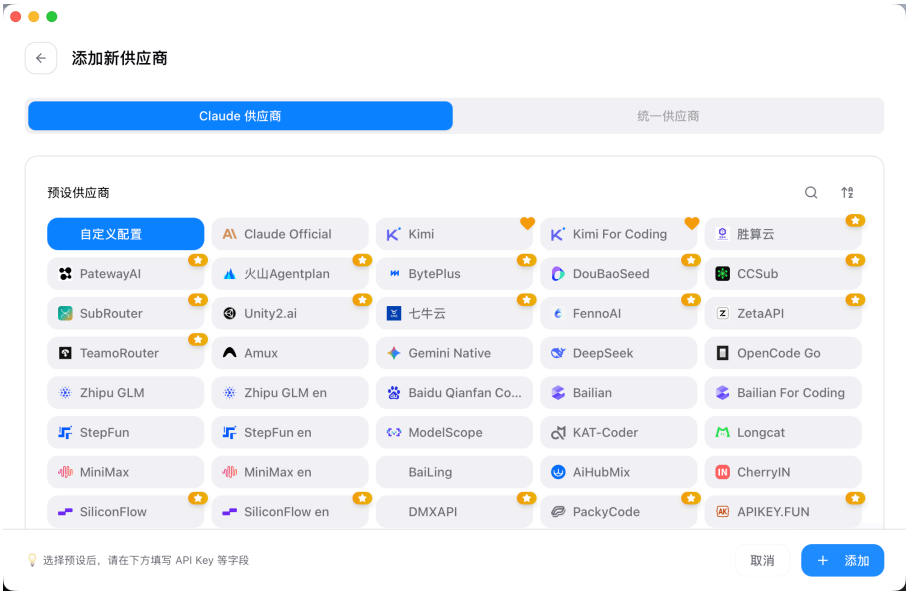


图 9　cc switch 添加新供应商的界面。

测试对话能得到一次稳定的回复，说明配置完成。配置生效后，Claude Code 的启动界面会显示当前使用的模型，图中左侧显示的就是模型名称。



图 10　Claude Code 的启动界面，左侧显示当前使用的模型。

3.8　第一次 AI 对话

环境配好后，进行第一次 AI 对话，目标是让 AI 解释当前项目目录，并给出下一步建议。步骤是：

1. 进入项目目录。
2. 告诉 AI 这次练习的目标。
3. 要求 AI 先规划，不写代码。
4. 检查 AI 是否理解。

这次对话的产出是一份可以执行的计划。第一次对话先不写代码，重点确认 AI 理解了项目和目标。

小结

第二天的开发环境由 Claude Code 与 IDE 组成，AI 生成的结果一律在本地运行验证。Node 为 JavaScript 提供了浏览器之外的运行环境，前端项目借助 npm 安装依赖，启动开发服务器和构建发布文件，安装后要确认命令实际指向的程序。终端操作的三条规则是先定位，看完整输出，报错连同命令交给 AI。模型通过 cc switch 配置，配好后的第一次对话先让 AI 解释项目，再拿出计划。

04
和 AI 对话的标准姿势

4.1 坏 Prompt 的问题

说得越清楚，AI 返工越少。发给 AI 的提示词叫 **Prompt**。坏 Prompt 常见三种写法，每种对应一个问题：

坏 Prompt 的写法	问题
“帮我做个网页”	目标不清
“越酷越好”	风格不清
“直接给代码”	没有验收

目标不清，AI 只能自行推测要做什么。风格不清，页面风格完全由 AI 自行决定。没有验收标准，做完之后无法判断结果是否合格。

4.2 好 Prompt 的结构

好 Prompt 把任务交代完整。以个人招聘主页的视觉美化为例，一个好 Prompt 分五个部分：

部分	例子里的写法
角色设定	让 AI 扮演一位专注于产品和个人品牌的设计师
任务目标	不改变现有页面结构和内容，只从视觉角度美化个人招聘主页
当前状态	页面有简介与项目，技能与联系方式四个板块，目前只有基础的 HTML 和 CSS 结构
美化要求	共六条，覆盖风格与排版，色彩与布局，细节与质感
输出要求	描述视觉优化的具体方向和细节，不写代码，只描述视觉效果和设计逻辑，并说明方案如何提升招聘方的阅读体验

其中的美化要求写到了可以直接执行的程度：

美化要求	例子里的写法
风格定义	极简专业风，参考 Apple 等官网的设计语言，追求克制的高级感
排版	制定清晰的字体层级，使用系统原生字体，确保跨平台显示一致，并调整行高和字间距，增加呼吸感
色彩系统	背景用柔和的浅色代替纯白以减轻视觉疲劳，文字用深灰代替纯黑，再选定一种主色调用于链接和按钮的悬停效果
空间与布局	大量留白，内容区域居中并限制宽度以减轻长文本的阅读压力，项目卡片对齐以保持视觉平衡
细节打磨	链接加下划线动画，项目卡片加细微阴影和鼠标悬停时的上浮效果，技能标签用浅色背景配深色文字并做圆角处理
质感	用细边框区分区域，增加精致度

五个部分写全了，AI 就不需要自行推测任务的细节。

4.3 概念与报错的提问

遇到不会的概念，提问时要说明希望 AI 解释到什么程度：

> 我不懂这个概念。请先用大二学生能理解的话解释，再给一个前端里的例子，最后告诉我是否需要现在掌握。

这段提问规定了解释的深浅，要求结合前端场景举例，并让 AI 判断这个概念现在是否需要掌握，避免在暂时用不到的知识上花太多时间。

遇到报错，要把完整报错连同命令和上下文一起交给 AI：

> 这是完整报错和我刚执行的命令。请先判断错误类型，再列出 3 个可能原因，最后只做最小修改。

这段提问给出了完整报错和刚执行的命令，并规定了处理顺序：先判断错误类型，再列出可能原因，最后只做最小修改，避免 AI 为了修一个错误大范围改动代码。

4.4 先规划后执行

复杂任务先进入**规划模式**，确认方案后，再让 AI 动手写代码。规划模式下 AI 只讨论方案，不修改文件。

规划时同样要说清结果并交代现场，还要多加一条要求：看不懂 AI 的改动时，先让 AI 解释，再执行。这样可以在动手之前发现方案里的问题，减少返工。

4.5 提示词工作流

提示词不是玄学，而是一套工作流，分五步：

1. 目标：说清要交付什么，给谁用，做到什么程度。
2. 背景：交代项目状态，报错和已有文件。
3. 限制：给出边界，说明哪里不要改，不随意添加依赖。
4. 验收：说明怎样检查算完成，要求 AI 给出检查方式。
5. 复盘：回顾这次对话里做对和做错的地方，把有用的经验留下来，下次复用。

每一次高质量的对话，都在减少下一次的返工。

小结

把要求说清楚，是减少 AI 返工的前提。坏 Prompt 的问题在于缺目标，缺风格，缺验收，好 Prompt 则用五个部分把任务交代完整。不懂概念时说明解释的深浅，遇到报错时附上命令与上下文，并限定只做最小修改。复杂任务先在规划模式里确认方案，看不懂的改动先让 AI 解释。写提示词时按五步工作流组织，并在最后复盘。

05
第一次前端实操：个人主页

5.1 实操目标与流程

第一次前端实操的目标，是做出一个能打开，能展示，能继续修改的作品：一个面向招聘或作品展示的个人主页单页。实操的产出是一个个人主页 Demo，过程分五步：

1. 描述想要的页面，确定内容模块。
2. 让 AI 生成首版代码。
3. 打开本地预览。
4. 用 F12 检查问题。

5. 继续对话修改，迭代页面。

先让整个流程从描述到预览完整走通，再追求页面美观。

5.2 描述页面与生成代码

第一步，先让 AI 把页面想清楚，暂不写代码。提问时说明页面的目标和包含的模块，先要结构和文案：

```
请为我设计一个个人主页，目标是展示前端学习能力。页面包括 Hero、技能、项目、联系方式。先给结构和文案。
```

其中 Hero 指页面顶部的首屏区域。AI 给出结构和文案后，先确认内容模块是否合适，再进入下一步。

第二步，让 AI 生成首版代码。生成前告诉 AI 技术限制：先用 HTML，CSS 和 JS 写，不引入复杂框架。首版代码只求最小可用，样式和细节留到后面的迭代里完善。

5.3 预览与检查

第三步，打开预览。单页作品可以直接打开 `index.html`：

```
$ open index.html
```

`open` 是 MacOS 终端里的命令，会用默认浏览器打开页面。Windows 上可以在文件夹里双击 `index.html`。也可以用 VsCode 的 Live Server 插件预览，或者启动 dev server 预览。

能看到页面，第一轮验收才刚开始。第四步，按 F12 打开开发者工具检查问题：Console 里没有报错，页面布局没有错乱，才算通过第一轮验收。

5.4 继续对话修改

第五步，在页面能打开的基础上继续对话修改，并让 AI 先说明改动范围：

```
页面已经能打开。请帮我改善层级和间距，不要增加新功能。修改前先说明你要改哪几个区域。
```

这段提问只针对层级和间距，明确不增加新功能，并要求 AI 修改前先说明要改哪几个区域，便于判断改动有没有超出范围。每轮修改只针对一个方向，改动范围小，出了问题也容易定位。

5.5 页面验收清单

页面好不好，不只看能不能打开。交付前按五项检查：

1. 标题是否清楚。
2. 模块是否完整。

3. 移动端是否可看。

4. 按钮是否可点击。

5. 内容是否像真实作品。

移动端的显示效果，可以在开发者工具里切换成手机尺寸查看。这份清单的用途是判断 AI 的输出能不能交付，不需要背下来。

5.6 理解代码结构

不要只拿结果，还要让 AI 解释每个文件与每个模块的作用，以及关键样式为什么这样写。

第一个前端作品完成后，下一步不是换工具，而是把作品整理成项目。整理或接手一个项目时，从三点入手：

关注点	做法
入口	先找 `main`，`App` 与 `package.json`
结构	看清组件，样式与工具函数分别放在哪里
风险	不要一次让 AI 大面积重写

单页作品的入口是 `index.html`。整理成项目后，入口通常是 `main` 或 `App` 文件，项目的依赖和常用命令记在 `package.json` 里。作品能运行，结构也看懂了，再进入下一轮修改。

小结

个人主页的实操分五步推进，从描述页面开始，到继续对话修改结束。首版代码先限定 HTML，CSS 和 JS，只求最小可用。第一轮验收以浏览器开发者工具的检查结果为准，没有报错，布局正常才算通过。之后按单一方向逐轮修改，交付前对照五项清单检查。作品完成后让 AI 讲解代码结构，为整理成项目做准备。

06
前端框架入门

个人主页实操只用 HTML，CSS 和 JS 写成单页，没有引入框架。页面增多，交互变复杂以后，就需要借助**前端框架**来组织代码。框架的作用，是让页面在长期修改中依然容易维护。

6.1 组件与状态

页面数量增加，交互变得复杂，需要跟踪的状态也越来越多。这时，直接用 HTML，CSS 和 JavaScript 编写的原生写法会越来越难维护。框架从两个方向解决这个问题：拆分组件与管理状态。

拆分组件，是把页面拆成一个个可以单独维护的部件，按钮，卡片和表单等元素都可以做成可复用的组件。组件的作用类似零件：一个页面由许多组件组合而成，同一个组件可以在多个位置重复使用，修改这个组件，所有用到该组件的位置都会一起更新。

管理状态，是让数据的变化稳定地反映到页面上。**状态**指页面当前持有的数据。以 Todo 网页为例，任务列表就是状态，新增，完成或删除一条任务都会改变这份数据，页面显示的内容要随之更新。框架负责让数据和页面保持一致，开发者不必每次手动修改页面。

让 AI 编写框架项目之前，先想清楚页面由哪些组件组成，哪些数据会发生变化。

6.2 框架的选择

常用的前端框架有 Vue，React 和 Next.js，按侧重点可以分成两类。

Vue 与 React	Next.js
更偏前端组件	页面和路由更完整
适合学习交互	接近产品开发
配套的库和工具成熟	具备全栈能力

路由指网址与页面之间的对应关系。全栈指前端与后端都能开发。Vue 与 React 适合先学组件和交互，Next.js 适合开发接近完整产品的项目。

搭建 Vue 或 React 项目时，还会遇到这几个名字：

```
Vite npm/pnpm react/vue
```

Vite 是前端构建工具，用来创建项目和启动本地开发服务器。npm 和 pnpm 是包管理工具，用来安装项目依赖，两者选一个使用。react 和 vue 是两个框架在 npm 上的包名。

6.3 Vue 与项目初始化

Vue3 的定位是渐进式前端框架。渐进式指可以先在页面的一小部分用上框架，再逐步扩大使用范围。Vue3 用接近 HTML 的模板描述页面，写法直观，学习曲线相对平滑，适合初学者入门，用来理解组件以及数据和页面的绑定。绑定指数据改变时页面自动跟着更新，这就是管理状态要做的事。

初始化一个 Vue3 项目用四条命令。命令前的 $ 代表终端提示符，输入时省略。

```
$ npm create vite@latest vue-demo
$ cd vue-demo
$ npm install
$ npm run dev
```

第一条命令用 Vite 创建名为 `vue-demo` 的项目，运行过程中命令行会询问使用哪个框架，Vue3 项目选择 Vue。第二条进入项目目录，第三条安装依赖，第四条启动项目，终端里会显示本地预览的地址。

执行命令时按终端操作的三条规则：先确认当前目录，执行后看完整输出，出现报错就把报错连同命令一起交给 AI。项目在本地成功运行之前，不急着修改代码。

6.4 React 与入口文件

React 强调用 JavaScript 描述界面，是很多公司前端项目的主流选择。React 的定位是组件化 UI，围绕 React 已经形成大量配套的库和工具，适合产品开发。

初始化 React 项目的命令与 Vue3 相同，只把项目名换成 `react-demo`，询问框架时选择 React。

```
$ npm create vite@latest react-demo
$ cd react-demo
$ npm install
$ npm run dev
```

项目运行起来以后，让 AI 解释 `src` 目录里的入口文件。入口文件是页面开始组装的地方，读懂入口文件，就能看清页面由哪些组件组成。

6.5 Next.js 与框架对照

Next.js 的定位是产品级 React 框架。除了界面，Next.js 还提供路由，服务端渲染和接口这些开发产品需要的能力。服务端渲染指页面先在服务器上生成，再发送给浏览器。接口即 API，是前端与后端之间请求与响应的约定。Next.js 能力更强，需要理解的概念也更多，初学阶段先用 Vue 或 React 理解组件，再接触 Next.js 更容易。

框架	定位	特点
Vue3	渐进式前端框架	模板直观，学习曲线平滑，适合入门
React	组件化 UI	用 JavaScript 描述界面，配套的库和工具多
Next.js	产品级 React 框架	提供路由，服务端渲染和接口，接近产品开发

三个框架都适合在项目里持续开发。选择哪一个，取决于当前任务是学习组件，还是开发完整的产品。

6.6 读懂项目目录

AI 生成一个框架项目后，开发者先看这几个文件和目录：

文件或目录	看什么
`package.json`	项目依赖和可运行的命令，`npm run dev` 里的 dev 就定义在这里
`src/main` 或 `src/App`	页面入口，页面从这里开始组装
`components` 目录	拆分出来的各个组件
`styles` 或 CSS 文件	页面样式
`README`	项目说明，例如安装与运行方法

看不懂目录时让 AI 讲解，可以用这条 Prompt：

> 请用表格解释这个前端项目的目录结构。告诉我哪些文件是入口，哪些文件可以先不看。

这条 Prompt 要求 AI 用表格回答，并区分入口文件和暂时可以不看的文件，读者可以先掌握项目的主干。读懂目录，是接手一个项目的第一步。

小结

页面一旦复杂，就要靠组件拆分与状态管理来保持可维护。入门阶段用 Vue 或 React 学组件和交互，需要路由与接口等产品能力时再考虑 Next.js。新项目用 Vite 初始化后，先在本地运行成功，再动手修改。接手 AI 生成的项目，先查清依赖，入口与组件分别在哪里。

07
界面生成与 Todo 项目

v0.dev 这类工具可以根据 Prompt 生成界面，但运行，调试和维护代码的工程工作仍由开发者完成。Todo 网页是第一个完整项目，需要分步推进，每一步验收，大改之前保存版本。

7.1 界面初稿与快速原型

v0.dev 适合生成界面初稿。开发者输入 Prompt 后，v0.dev 生成页面并显示预览，同时给出对应的代码。对预览不满意，就修改 Prompt 重新生成，这个过程可以反复进行，直到得到满意的原型。



图 11　v0.dev 的对话界面，中间一栏是 Prompt 与生成过程，右侧是页面预览区。

图中输入的是一份招聘用个人主页的需求，中间一栏列出 v0.dev 的生成过程，右侧黑色区域在生成完成后显示页面预览。

生成结果只是界面初稿。把代码导出到本地后，开发者仍要运行项目，读懂结构，再继续迭代。

7.2　好页面 Prompt 的要素

一个好的页面 Prompt 要交代三个要素。以招聘用的个人主页为例，三个要素可以这样写：

要素	写什么	个人主页的写法
页面类型	个人主页，官网或后台	招聘用的个人主页
视觉风格	简洁，科技或课程风格	简洁
内容结构	有哪些模块，模块的优先级	Hero 首屏在前，其后是技能与项目，联系方式放在最后

记这三个要素时，先分清各自管什么，再用一个具体页面把三者串起来。

用 v0.dev 生成个人主页时，按四步进行：

1. 写页面结构。
2. 指定视觉风格。
3. 指定内容模块。
4. 导出代码。

导出的代码同样要在本地运行验证。

7.3 Todo 网页的迭代

第一个完整项目是 Todo 网页：做一个能新增，完成和删除任务的 Todo，产出一个 Todo Demo。推进的原则是先做出最小可用的版本，再增加交互，然后优化样式，最后写 README。最小可用指功能最少但能正常使用的版本。

Todo 不一次做完，而是分五步让 AI 逐步推进：

1. 做静态页面，先把结构和样式做出来，此时还不能操作。
2. 加上新增任务的功能。
3. 加上完成和删除任务的功能。
4. 优化样式。
5. 写 README。

AI 可以参与每一步，但每一步都要由人验收：这一步的功能可以正常使用，页面没有报错，才进入下一步。

7.4 版本回滚

每一次大改之前都要保存版本。出错可以修正，更大的风险是改坏以后回不到之前能用的状态。回到之前保存的版本，叫作**版本回滚**。有了可以回滚的版本，开发者才敢放手尝试。

保存版本用第一天学过的 Git。项目开始时执行一次 `git init`，之后每次大改之前执行 `git add -A` 和 `git commit -m "说明"`，生成一个存档点。改坏了，就让 AI 把项目恢复到上一次提交的状态。Todo 的每一步开始前都提交一次，哪一步改坏了，就退回上一步的提交。

小结

v0.dev 负责快速给出界面初稿，Prompt 写清页面类型，视觉风格与内容结构，生成结果才贴近需求。Todo 网页按五步分轮完成，每轮先验收再继续。每次大改前用 Git 提交，改坏后可以退回上一个存档点。

08
Claude Code 进阶工作流

进阶用法的目标，是把与 Claude Code 的一次次对话变成可控的开发过程。开发者要选对工作模式，管理好终端命令与后台任务，控制上下文的长度，并把固定动作与复杂任务交给专门的机制处理。

8.1 Claude Code 的工作模式

Claude Code 有三种模式。默认模式用于对话，规划模式用于规划，自动模式用于执行。输入框下方的提示文字显示当前所处的模式：

模式	提示	关注点
默认	`? for shortcuts`	修改文件前一定询问用户
规划	`plan mode on`	只讨论，不修改文件
自动	`accepts edits on`	自动修改文件

按 Shift+Tab 可以在三种模式之间轮流切换。

跨多个文件，影响较大的任务，先在规划模式里让 AI 给出方案，开发者确认方案后，再切换到默认或自动模式执行。

8.2 终端命令与后台任务

安装依赖和启动项目，运行测试和查看日志，这类命令都适合交给 AI 执行。命令由 AI 执行，开发者仍按终端操作的三步检查：先定位，再执行，最后看输出。命令的作用范围通常由当前目录决定，在错误的目录里执行，结果会落到错误的位置。只看最后一行容易漏掉前面的报错原因，所以要看完整的终端日志。

有些命令执行完就结束，开发服务器则会持续运行，属于后台任务。前端项目用 `npm run dev` 启动的就是开发服务器，英文称为 dev server。管理开发服务器要掌握三件事：是否启动成功，端口是多少，以及如何停止。

启动成功时，终端会打印一个本地访问地址，地址中冒号后面的数字就是端口，在浏览器中打开这个地址即可预览页面。在运行开发服务器的终端里按 Ctrl+C 可以停止服务器。由 AI 在后台启动的开发服务器，可以直接让 AI 停止。

8.3 上下文与项目记忆

对话太长会影响 AI 的判断。处理方法是阶段性地总结，让 AI 只保留关键的上下文。Claude Code 为此提供两个命令。`/compact` 把之前的对话压缩成一份总结，保留要点后继续工作。`/clear` 清空当前对话，从头开始，适合切换到新任务时使用。

压缩时，总结里要保留交代任务的三类信息：说清结果，交代现场和给出边界。清空之后，AI 不再记得之前的对话，这三类信息需要重新交代。

每开一次新对话都重新交代项目规则，既费时又容易遗漏。**项目记忆文件**解决这个问题：把项目规则，常用命令和目录说明写进一个文件，AI 每次进入项目都会先读。Claude Code 的项目记忆文件是项目根目录下的 `CLAUDE.md`，可以写入启动命令 `npm run dev`，组件统一放在 `components` 目录，不随意增加依赖这类约定。

8.4 Hook 与 Sub Agent

Hook 把固定动作自动化，例如改完代码后自动格式化，自动运行检查和提醒风险。这些动作每次都要做，交给 Hook 以后，到了设定的时机就会自动执行，不需要每次提醒 AI。Hook 在 Claude Code 的设置中配置，写明在什么时机运行什么命令。

Sub Agent 用来把复杂任务拆分出去。主对话负责决策，子任务负责搜索，检查和生成局部方案，完成后把结果交回主对话，由主对话决定下一步。这样分工，主对话可以专注于决策。

小结

规划模式先出方案，默认模式逐个确认修改，自动模式直接修改，三种模式用 Shift+Tab 切换。终端命令可以交给 AI 执行，开发者负责确认目录与阅读完整日志，并清楚开发服务器的端口与停止方法。上下文过长时用 `/compact` 压缩或用 `/clear` 清空，长期有效的规则写进 `CLAUDE.md`。每次都要做的固定动作交给 Hook，可以拆开的复杂任务交给 Sub Agent。

09
Skill 机制

Skill 机制解决的问题是：一次做对的方法，怎样在以后的同类任务里稳定地重复。办法是把这套方法写成 Skill 文件，交给 AI 在同类任务中遵循。

9.1 Skill 的含义

Skill 是写给 AI 的“专业工作说明书”。一份 Skill 写清三件事：

部分	写什么
触发条件	什么时候应该使用这套方法
执行流程	先做什么，后做什么，哪里要谨慎
质量标准	输出做到什么程度才算合格

普通 Prompt 只在当前对话里起作用。同一套方法写成 Skill 以后，可以在不同的对话和任务里反复使用。

9.2 Skill 的作用与本质

同一件事反复向 AI 解释，是 AI Coding 中不容易察觉的浪费。有没有 Skill，区别在于规则放在哪里。

情况	做法	结果
没有 Skill	每次重新说明风格，流程和限制	容易遗漏或偏离要求，返工多
有 Skill	规则固定在文件里，AI 先读再做	同类任务的输出更稳定

Skill 的作用不是让 AI 更聪明，而是让 AI 更稳定地按开发者的标准做事。

Skill 不是知识库，而是**可复用上下文**。上下文指 AI 处理任务时能看到的信息。知识库存放的是资料，Skill 存放的是做事的方法：Skill 把“怎么做”从聊天记录里提取出来，变成项目的固定能力，需要时加载进上下文，在不同对话里重复使用。

Skill 记录的是稳定的流程，质量标准和项目经验。Skill 不是一次性的灵感，也不是一句“万能咒语”，不能替代开发者的判断。

9.3 Skill 的目录与内容

一个 Skill 就是一个文件夹，核心入口是说明文件 `SKILL.md`，旁边可以放参考资料，脚本和素材。典型的 Skill 文件夹结构是：

```
my-skill/
├── SKILL.md
├── scripts/
├── references/
├── assets/
└── ...
```

文件或文件夹	内容
`SKILL.md`	说明文件，也是入口
`scripts`	可复用的脚本
`references`	参考资料
`assets`	素材
`...`	其他需要的文件

`SKILL.md` 开头写 Skill 的名称和描述，AI 根据描述判断当前任务是否需要这个 Skill，触发条件就是通过这段描述起作用的。

`SKILL.md` 里通常写五项内容：

1. 什么时候使用。
2. 工作步骤。
3. 质量标准。

4. 示例输入输出。

5. 可复用脚本或素材。

前三项分别对应触发条件，执行流程和质量标准。示例输入输出给 AI 一个可以参照的样例。可复用的脚本或素材放在对应的文件夹里，需要时由 AI 读取或运行。

9.4 Skill 与相近概念

Skill 容易和 Prompt 与 Sub Agent 混淆，三者的区别是：

对象	作用	特点
Prompt	解决当前问题	一次性，用完容易散落在聊天记录里
Skill	定义一类任务的做事方法，提升输出的稳定性	可复用，能持续改进，像操作手册
Sub Agent	把任务拆出去执行，提升并行度	几件事可以同时进行，像临时助手

工具，Skill 和 Plugin 也常放在一起出现，三者各管一层：

名称	解决什么	说明
工具	提供能力	搜索和读文件，生成图片和运行命令
Skill	提供方法	告诉 AI 遇到某类任务时怎样做才稳妥
Plugin	打包成套	把工具，Skill 和配置组合成可以安装的整体

把三者分清楚，后面接入更多能力时就不会混乱。

9.5 前端 Skill 的拆解与应用

前端开发中，Skill 常用来做四类事：

1. 统一 UI 风格。

2. 检查移动端适配。

3. 生成组件规范。

4. 修复常见报错。

拆解一个具体的 Skill，要看清三点：怎样触发，怎样约束输出，怎样验证结果。阅读时对照触发条件，执行流程和质量标准三部分。拆解与参考可以从这些资源入手。

拆解与参考	网页动效组件
impeccable	originkit
taste skill	react bits
shadcn	unicorn
ui-ux-pro-max	
design MD Collection	

另一个可以拆解的例子在 `https://getdesign.md/apple/design-md`。

用 Skill 改进 Todo 网页，是让 AI 按固定的前端规范优化项目，分四步：

1. 接入 Skill。
2. 提出优化目标。
3. 让 AI 先规划。
4. 检查结果。

接入 Skill，指把 Skill 文件夹放到 Claude Code 能读取的位置，例如项目里的 `.claude/skills` 目录。第三步可以使用规划模式，开发者确认方案后再让 AI 修改。四步完成后，得到一个更稳定的 Todo 项目。

小结

Skill 用触发条件，执行流程和质量标准三部分，把一次做对的方法固定下来。Skill 以文件夹的形式存放，入口是 `SKILL.md`，AI 依据其中的描述决定何时使用。Prompt 只服务当前问题，Skill 面向一类任务，Sub Agent 用来分担任务。工具提供具体能力，Plugin 负责把工具与 Skill 打包安装。前端开发中，Skill 常用于统一界面风格与组件规范，也用于检查移动端适配和修复常见报错。

10
总结与课后练习

AI 写代码的速度很快，开发者的价值在于验收结果，避开常见错误，并按固定的工作流推进开发。课后练习是做一个智能客服官网，目标与各项要求列在讲义末尾的课后作业中。

10.1　验收与常见错误

速度不等于质量。AI 生成代码以后，开发者要做四项验收：

1. 打开页面查看效果。

2. 运行项目。
3. 阅读报错信息。
4. 检查哪些文件被修改。

使用 AI 开发时，有五个常见错误，每个错误都有对应的做法：

常见错误	正确做法
复制自己看不懂的代码	先让 AI 解释，看懂再用
一次让 AI 做太多事	一次只改一个方向
忽略报错	把报错连同命令一起交给 AI
没有版本回滚	大改前用 Git 提交
把生成当作完成	生成之后逐项验收

10.2 完整的工作流

第二天的内容可以归结为一条工作流：

1. 描述任务。
2. 生成代码。
3. 本地预览。
4. 定位问题。
5. 继续迭代。
6. 把经验写成 Skill。

执行这条工作流时遵守三条原则：先运行起来，再读懂结构，每次只做小步修改。

Vibe Coding 的核心，是开发者用工程方法管理 AI 的输出，衡量进步的标准不是 AI 写了多少代码。每完成一次开发，就把其中做对的方法整理成 Skill，供下一次开发直接使用。

小结

AI 生成代码之后，开发者要亲自查看运行效果和文件改动，不能把生成当作完成。避免五个常见错误，要做到看懂再用，小步修改和保留版本。当天的工作流从描述任务开始，到把经验写成 Skill 结束。

练习与作业

复习题

1. 说明在 Vibe Coding 中把 AI 当作开发搭档与当作搜索框的区别，并写出把 AI 当实习生时要交代清楚的四件事。
2. 说明前端，后端和 API 各自负责什么，并写出网页三层结构中 HTML，CSS 和 JavaScript 各自的作用。
3. 比较 IDE 与 CLI 两种使用方式，并写出检查 Node 和 npm 是否安装成功，以及查看命令实际指向哪里的命令。
4. 说明“帮我做个网页”“越酷越好”“直接给代码”三种说法各自的问题，并写出遇到报错时向 AI 提问要附上的内容和提出的要求。
5. 写出个人主页实操的五个步骤，说明页面打开以后满足哪些条件才算通过第一轮验收，并写出 F12 里 Elements，Console 和 Network 三个区域分别看什么。
6. 说明前端框架靠哪两件事让页面能被长期维护，并比较 Vue，React 与 Next.js 的侧重点。
7. 写出初始化一个 React 项目的四条命令，并说明 AI 生成项目以后先看的五个文件或目录各看什么。
8. 写出好页面 Prompt 的三个要素和 Todo 网页的五步迭代路线，并说明每次大改前保存版本的原因和做法。
9. 比较 Claude Code 默认，规划和自动三种模式，说明哪类任务要先规划，并写出管理开发服务器要掌握的三件事。
10. 说明对话太长时压缩与清空上下文的做法，并说明项目记忆文件，Hook 和 Sub Agent 各自解决什么问题。
11. 说明一份 Skill 写清哪三件事，并比较 Skill 与 Prompt，Sub Agent 和 Plugin 的区别。
12. 写出第二天工作流的六个环节，以及使用 AI 开发时要避免的五个常见错误。

课后作业

课后练习是做一个智能客服官网。官网要能展示产品价值，引导用户咨询，并能在手机上正常浏览。作业从页面结构，交互功能和交付物三方面提出要求。

页面结构

官网包含五个部分：

1. 首页 Hero。
2. 产品能力介绍。
3. 适用场景。

4. 价格或套餐。
5. 联系方式。

交互功能

官网实现四项交互：

1. 导航跳转。
2. 表单输入。
3. FAQ 展开收起。
4. 移动端适配。

导航跳转指点击导航栏里的栏目，页面跳到对应的区块。FAQ 是常见问题列表，展开收起指点击一个问题显示答案，再点一次收起。移动端适配指页面在手机的小屏幕上自动调整布局。

交付物

提交四样东西：

1. 可运行项目。
2. README。
3. 截图。
4. 一段使用 AI 的过程记录。

三份清单用于验收，提交前逐项对照，判断作品能否交付。